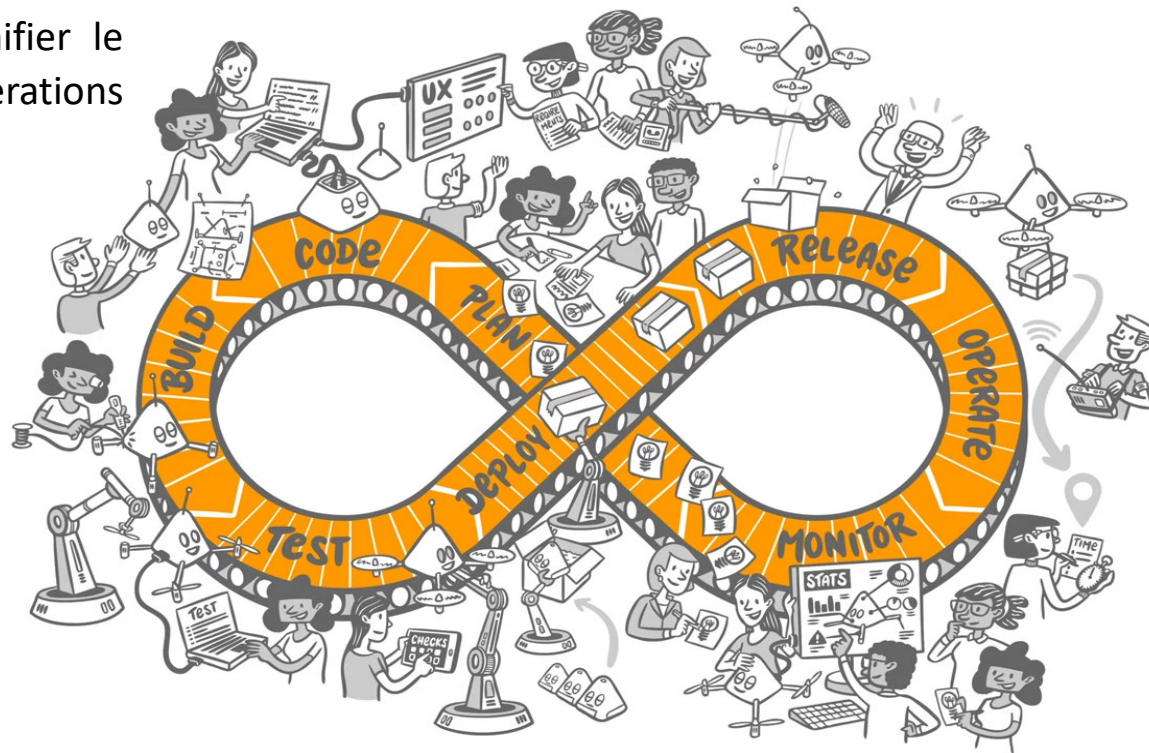


DevSecOps au sein de l'écosystème GitHub

Le **DevOps** est une culture et une pratique logicielle qui vise à unifier le développement (**Dev**) et les opérations informatiques (**Ops**).



DevOps à DevSecOps

Dans le modèle DevOps traditionnel, la sécurité intervient souvent à la fin du cycle (étape de "**Release**").

Exemple d'incident:

le code source de Twitch qui a fuité sur Internet en 2021. Contenait des **clés API** en clair, des **tokens d'accès** et des **secrets** de production directement **commités** dans le code!!!!

Motivation :

Détecter une vulnérabilité au stade de l'écriture du code ne requiert qu'un investissement temporel de l'ordre de quelques minutes, alors que remédier à cette même faille après une exploitation malveillante peut occasionner des préjudices financiers s'élevant à plusieurs millions.

DevOps à DevSecOps

Dans le modèle DevOps traditionnel, la sécurité intervient souvent à la fin du cycle (étape de "**Release**").

Le **DevSecOps** est l'intégration naturelle de la sécurité dans la boucle DevOps. L'idée fondamentale est que **la sécurité est une responsabilité partagée** dès le premier jour.

DevSecOps consiste à intégrer les tests de sécurité le plus tôt possible dans le cycle de vie du développement (dès la phase de design et de code), plutôt que d'attendre la mise en production.

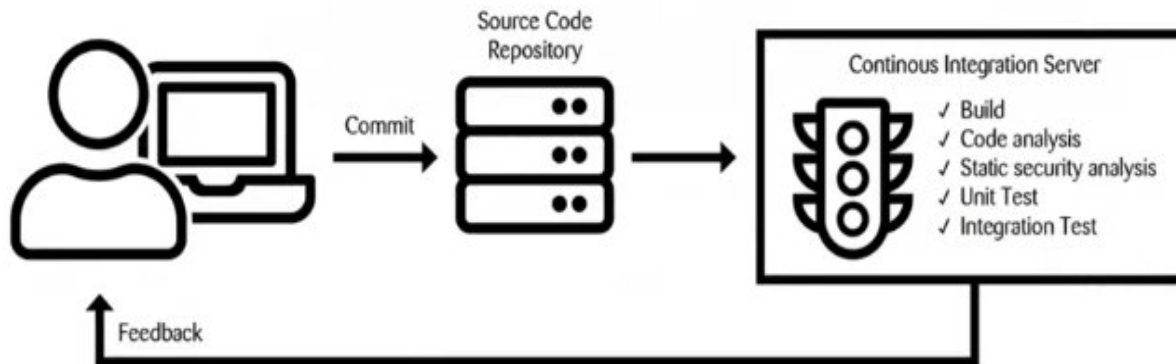
Avantages :

- **Réduction des coûts** : Corriger une faille en phase de code coûte beaucoup moins cher qu'en production.
- **Rapidité** : Automatiser la sécurité permet de ne pas ralentir les déploiements.
- **Conformité continue** : On ne vérifie plus la sécurité une fois par an, mais à chaque "**commit**".

DevSecOps et le Pipeline CI-CD

Le pipeline CI/CD permet de mettre en œuvre la philosophie DevSecOps.

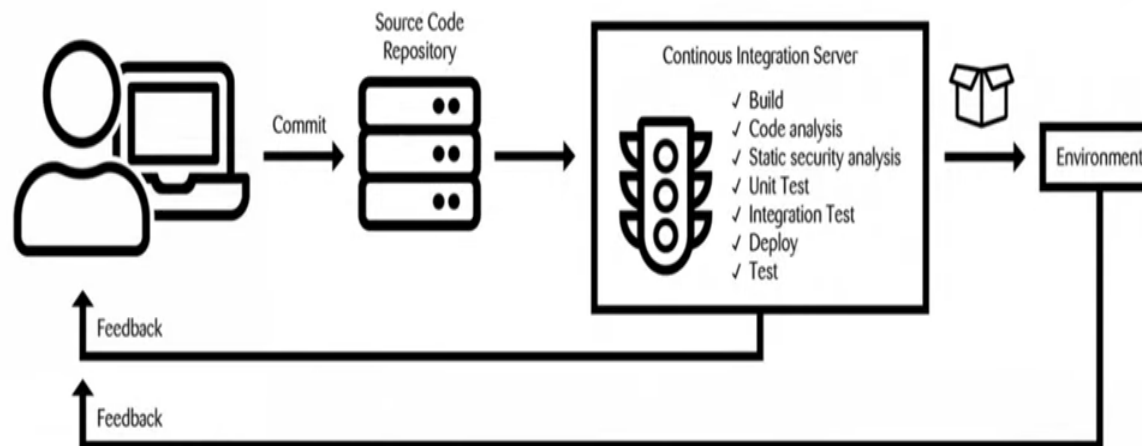
Continuous Integration



DevSecOps et le Pipeline CI-CD

Le pipeline CI/CD permet de mettre en œuvre la philosophie DevSecOps.

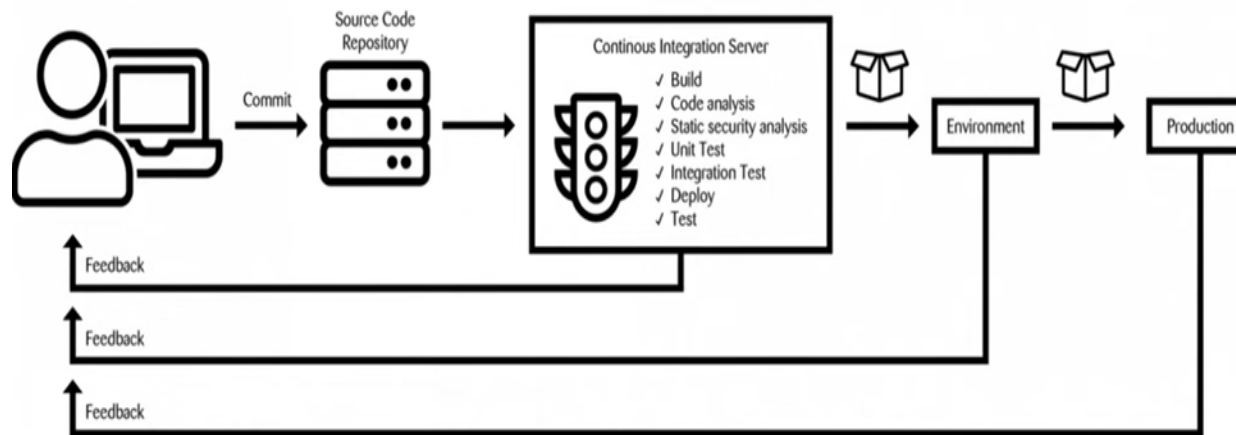
Continuous Delivery



DevSecOps et Le Pipeline CI-CD

Le pipeline CI/CD permet de mettre en œuvre la philosophie DevSecOps.

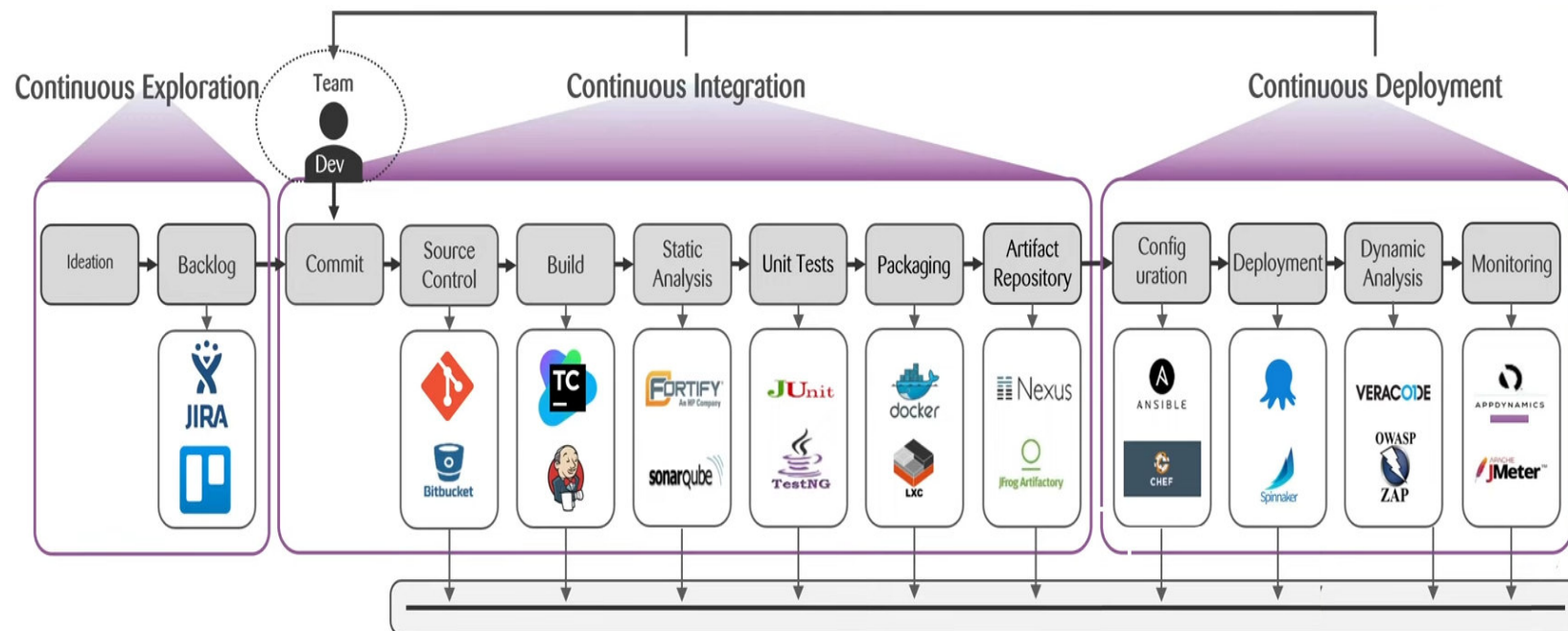
Continuous Delivery/Deployment



DevSecOps et Le Pipeline CI-CD

Le pipeline CI/CD permet de mettre en œuvre la philosophie DevSecOps.

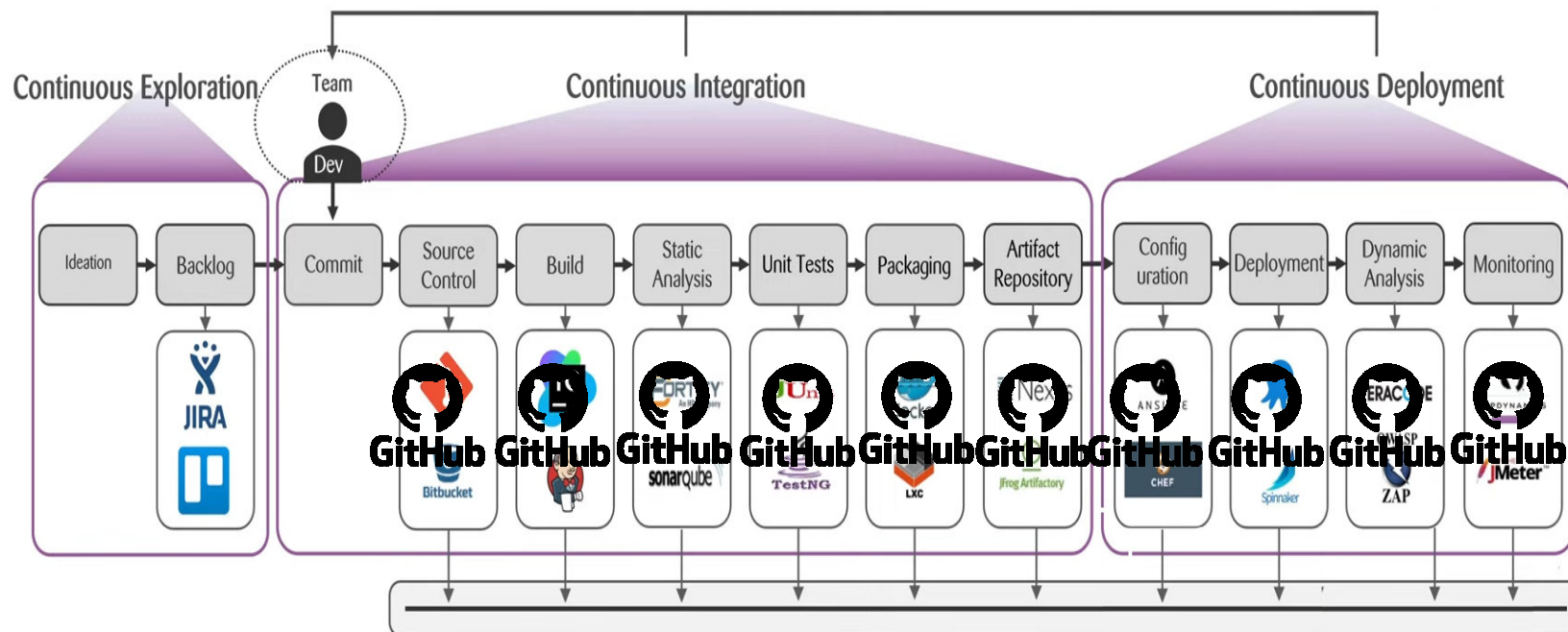
CI-CD en pratique



DevSecOps et Le Pipeline CI-CD

Le pipeline CI/CD permet de mettre en œuvre la philosophie DevSecOps.

CI-CD en pratique avec GitHub



DevSecOps et Le Pipeline CI-CD

Le pipeline CI/CD permet de mettre en œuvre la philosophie DevSecOps.

CI-CD en pratique avec GitHub

Pourquoi DevSecOps avec GitHub ?

GitHub est passé d'une simple plateforme d'hébergement de code à une plateforme de développement **complète** intégrant nativement la sécurité.

AVANTAGE: Simplification de la sécurité dans un écosystème GitHub

Dans GitHub, la **Pull Request (PR)** est le point central : avant qu'un code ne soit fusionné, des contrôles automatiques sont lancés. Ceci est réalisé grâce à **gitHub Actions**, une fonctionnalité d'intégration continue et de déploiement continu (CI/CD) directement intégrée à GitHub.

DevSecOps et Le Pipeline CI-CD

Le pipeline CI/CD permet de mettre en œuvre la philosophie DevSecOps.

CI-CD en pratique avec GitHub

AVANTAGE: Simplification de la sécurité dans un écosystème GitHub

Outil GitHub	Exemple de Fonction DevSecOps	Type de Scan
Dependabot	Analyse les dépendances (bibliothèques) et crée des PR automatiques pour corriger les versions vulnérables.	SCA (Software Composition Analysis)
CodeQL	Analyse le code source pour trouver des erreurs de logique ou des failles (injections SQL, etc.).	SAST (Static Application Security Testing)
Secret Scanning	Détecte si des mots de passe, clés API ou jetons ont été poussés par erreur dans le dépôt.	Secret Detection
OWASP ZAP	Détecter les vulnérabilités (problèmes de configuration, logique métier...) qui ne peuvent pas être détectées par une analyse statique du code	DAST (Dynamic Application Security Testing)
GitHub Actions	Automatise tout le workflow (Build, Test, Scan, Deploy) avec tous les testes nécessaires.	CI/CD Orchestration

GitHub Actions et Le CI-CD

- GitHub Actions est un moteur d'automatisation. Il vous permet de créer des flux de travail personnalisés (appelés **Workflows**) pour compiler, tester et déployer votre code à chaque modification.
- **GitHub Actions** est l'outil d'intégration et de déploiement continu (CI/CD) natif de GitHub.
- Il permet d'automatiser les vérifications de sécurité directement dans le dépôt de code.

GitHub Actions et Le CI-CD

Vocabulaire associé à GitHub Actions

1. **Workflow** (Flux de travail) : C'est le processus automatisé global. Il est défini par un fichier de configuration au format **YAML** (situé obligatoirement dans le dossier **.github/workflows/** de votre projet).
2. **Event** (L'Événement) : C'est le déclencheur. Un workflow démarre en réponse à un événement, par exemple : un **push** sur la branche principale, la création d'une **pull request**, ou même une planification quotidienne.
3. **Job** (La Tâche) : Un workflow est composé d'un ou plusieurs jobs. Par défaut, les jobs s'exécutent en parallèle. Par exemple, nous pouvons avoir un Job pour le test SAST et un autre pour le DAST.
4. **Step** (L'Étape) : Chaque Job est une succession d'étapes. Une étape peut être une simple commande shell (ex: **npm install**) ou l'appel à une action pré-packagée (ex: télécharger le code, lancer un scanner de sécurité).
5. **Runner** (l'Exécuteur) : C'est le serveur physique ou la machine virtuelle qui exécute concrètement les instructions de votre workflow (**fourni** par GitHub ou **hébergé** par vos propres soins).

GitHub Actions et Le CI-CD

les trois analyses majeures que nous intégrons via GitHub Actions: (SAST, SCA, DAST)

- **SAST (Static Application Security Testing)** : L'analyse de la boîte blanche. L'outil lit votre code source (sans l'exécuter) pour y trouver des vulnérabilités (mots de passe codés en dur, injections SQL potentielles, etc.).
Outil courant : CodeQL
- **SCA (Software Composition Analysis)** : * Le concept : L'analyse de vos dépendances. Votre application utilise des bibliothèques externes (via *npm*, *pip*, *maven*). Le **SCA** vérifie si ces bibliothèques ont des failles de sécurité connues (CVE) répertoriées publiquement.
Outil courant : Trivy ou Dependabot.
- **DAST (Dynamic Application Security Testing)** : L'outil attaque votre application pendant qu'elle s'exécute en envoyant des requêtes malveillantes pour voir comment elle réagit.
Outil courant : OWASP ZAP.

GitHub Actions et Le CI-CD

Illustration de fichier de configuration YAML (**python-security-pipeline.yml**) de tests : SAST, DAST et SCA, sur un exemple simple en Python (**app.py**) qui utilise une bibliothèque vulnérable (**requirements.txt**).

Le fichier **python-security-pipeline.yml** doit être dans l'emplacement [.github/workflows/](https://github.com/actions/workflows/)

```
1 Flask==2.2.2
2 Werkzeug==2.2.2
3 # Vulnérabilité SCA : Composant tiers obsolète avec des CVE publiques
4 urllib3==1.26.4
```

requirements.txt

App.py

```
1 import os
2 from flask import Flask, request, abort
3 from markupsafe import escape
4
5 app = Flask(__name__)
6
7 # Jeton secret attendu pour accéder à l'administration
8
9 EXPECTED_TOKEN = "token_etudiant_123"
10
11 @app.route('/')
12 def home():
13     return "Bienvenue sur l'environnement DevSecOps !"
14
15 @app.route('/bonjour')
16 def bonjour():
17     nom = request.args.get('nom', 'Invité')
18     nom_securise = escape(nom) # Route sécurisée
19     return f"<h1>Bonjour {nom_securise}</h1>"
20
21 # --- NOUVELLE ZONE PROTÉGÉE ---
22 @app.route('/admin')
23 def admin_panel():
24     # 1. Vérification des privilèges (Authentication)
25     auth_header = request.headers.get('Authorization')
26
27     if auth_header != EXPECTED_TOKEN:
28         # Rejet de la requête si le scanner ou l'utilisateur n'a pas le bon jeton
29         abort(401, description="Accès refusé : Jeton d'authentification manquant ou invalide.")
30
31     # 2. Zone vulnérable (Accessible uniquement après authentification)
32     # Nous omettons volontairement la fonction escape() ici pour créer une faille XSS
33     action = request.args.get('action', 'affichage_dashboard')
34
35     return f"<h1>Administration</h1><p>Action exécutée : {action}</p><i>Données confidentielles du serveur...</i></p>"
36
37 if __name__ == '__main__':
38     app.run(host='0.0.0.0', port=5000, debug=False)
```

GitHub Actions et Le CI-CD

python-security-pipeline.yml

Les actions de test
sont déclenchées sur
push ou *pull_request*

```
1   name: Python DevSecOps Pipeline
2
3   on:
4     push:
5       branches: [ "main" ]
6     pull_request:
7       branches: [ "main" ]
8
9   jobs:
10    # -----
11    # JOB 1 : SAST (Analyse Statique du code source Python)
12    # -----
13    sast-scan:
14      name: Analyse Statique CodeQL
15      runs-on: ubuntu-latest
16      permissions:
17        security-events: write
18        actions: read
19        contents: read
20
21      steps:
22        - name: Récupération du code
23          uses: actions/checkout@v4
24
25        - name: Initialisation de CodeQL
26          uses: github/codeql-action/init@v4
27          with:
28            languages: python
29
30        - name: Exécution de l'analyse CodeQL
31          uses: github/codeql-action/analyze@v4
32
33    # -----
34    # JOB 2 : SCA (Analyse des dépendances Python)
35    # -----
36    sca-scan:
37      name: Analyse SCA Trivy
38      runs-on: ubuntu-latest
39
40      steps:
41        - name: Récupération du code
42          uses: actions/checkout@v4
43
44        - name: Scan du fichier requirements.txt
45          uses: aquasecurity/trivy-action@master
46          with:
47            scan-type: 'fs'
48            scan-ref: '.'
49            format: 'table'
50            exit-code: '0' # On ne bloque pas le pipeline
51            ignore-unfixed: true
52            vuln-type: 'library'
53            severity: 'HIGH,CRITICAL'
54
```

GitHub Actions et Le CI-CD

python-security-pipeline.yml

...suite

NB: Les informations sensibles nécessaires au déploiement ou aux tests doivent être stockées dans **GitHub Actions Secrets**.

Pour définir le **TOKEN** nécessaire au lancement de notre page Web par ZAP, allez sur **GitHub** : dans **Settings** > **Secrets and variables** > **Actions**. Ajoutez un secret nommé **ZAP-AUTH-TOKEN** avec la valeur *token_etudiant_123*

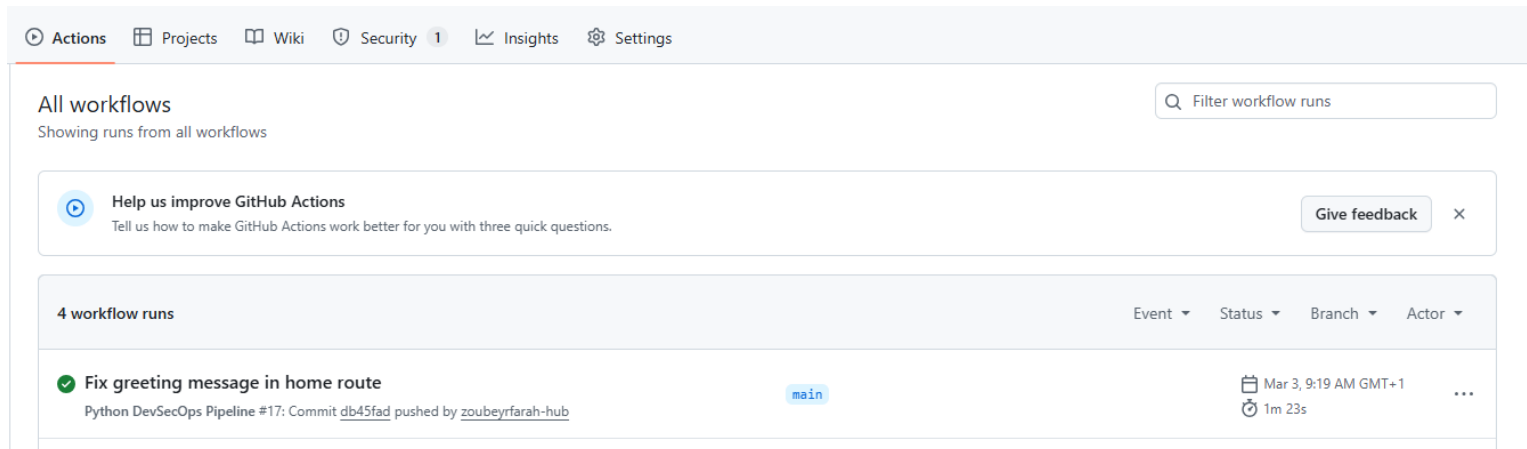
```
55 # -----
56 # JOB 3 : DAST (Analyse Dynamique Authentifiée)
57 # -----
58 dast-scan:
59   name: Analyse Dynamique ZAP avec Privilèges
60   runs-on: ubuntu-latest
61   permissions:
62     issues: write # Autorise ZAP à créer un ticket d'incident
63     contents: read # Autorise la lecture du code
64   steps:
65     - uses: actions/checkout@v4
66     - uses: actions/setup-python@v5
67       with:
68         python-version: '3.10'
69
70     - name: Installation et Démarrage
71       run: |
72         pip install -r requirements.txt
73         python app.py &
74         sleep 5
75
76     # --- AJOUT DE CETTE ÉTAPE POUR CORRIGER L'ERREUR DE PERMISSION ---
77     - name: Accorder les permissions d'écriture pour le conteneur ZAP
78       run: chmod 777 $GITHUB_WORKSPACE
79
80     - name: Scan OWASP ZAP (Authentié de manière sécurisée)
81       uses: zapproxy/action-baseline@v0.14.0
82       # On utilise les variables d'environnement prévues par ZAP pour éviter les problèmes d'espaces
83       env:
84         ZAP_AUTH_HEADER: "Authorization"
85         ZAP_AUTH_HEADER_VALUE: "${{ secrets.ZAP_AUTH_TOKEN }}"
86         ZAP_AUTH_HEADER_SITE: "http://localhost:5000"
87       with:
88         target: 'http://localhost:5000'
89         fail_action: false
```


GitHub Actions et Le CI-CD

Résultat du test

Une fois que le fichier YAML est poussé sur GitHub, la plateforme prend le relais.

1. Allez sur la page principale de votre dépôt GitHub.
2. Cliquez sur l'onglet "**Actions**" (situé en haut, sous le nom de votre dépôt).
3. Dans la barre latérale gauche, vous verrez la liste de vos workflows. Au centre, vous verrez la liste des exécutions (les "runs").
4. Cliquez sur l'exécution la plus récente



GitHub Actions et Le CI-CD

Résultat du test

Une fois dans une exécution spécifique, vous verrez un graphique visuel ou une liste de vos **Jobs** sur la gauche (ex: sast-scan, sca-scan, dast-scan).

•**Pastille Verte** : Le job a réussi. Aucune faille bloquante n'a empêché l'exécution.

•**Pastille Rouge** : Le job a échoué.

•**Action requise** : Cliquez sur un Job spécifique pour voir la console s'ouvrir. Vous pouvez y lire les logs ligne par ligne en temps réel. C'est ici que vous verrez, par exemple, le tableau généré par Trivy listant vos dépendances vulnérables.

The screenshot displays the GitHub Actions interface for a workflow named 'python-security-pipeline.yml'. The top navigation bar includes links for Code, Issues (2), Pull requests, Actions (selected), Projects, Wiki, Security (1), Insights, and Settings. Below the navigation bar, the workflow is identified as 'Python DevSecOps Pipeline' with a green checkmark and the title 'Fix greeting message in home route #17'. A summary section shows the workflow was triggered via push 2 weeks ago by 'zoubeyrfarah-hub' (commit db45fad on main), with a status of 'Success', a total duration of '1m 23s', and 1 artifact. A list of jobs is shown on the left: 'Analyse Statique CodeQL', 'Analyse SCA Trivy', and 'Analyse Dynamique ZAP avec Privilèges', all with green checkmarks. On the right, a detailed view of the 'python-security-pipeline.yml' workflow shows three jobs: 'Analyse Statique CodeQL' (51s), 'Analyse SCA Trivy' (23s), and 'Analyse Dynamique ZAP...' (1m 18s), all marked as successful with green checkmarks.